

Programação Paralela — Atividade 09

1) Explique detalhadamente o que é e para que se destinam:

a) POSIX

POSIX significa Portable Operating System Interface, ou seja, Interface Portável de Sistema Operacional. Trata-se de um conjunto de normas criado para padronizar a forma como programas interagem com sistemas operacionais, principalmente sistemas baseados em Unix e Linux.

O objetivo principal do POSIX é garantir portabilidade, isto é, permitir que um programa escrito seguindo esse padrão possa ser compilado e executado em diferentes sistemas operacionais compatíveis, com poucas ou nenhuma alteração no código-fonte.

O POSIX define elementos como chamadas de sistema, manipulação de arquivos, processos, sinais, threads, permissões, comandos de shell e utilitários básicos do sistema. Dessa forma, ele facilita o desenvolvimento de aplicações que precisam funcionar em vários ambientes.

Na programação paralela, o POSIX é importante principalmente por causa das POSIX Threads, também chamadas de pthreads. As pthreads permitem criar múltiplas threads dentro de um mesmo programa, possibilitando que diferentes partes do código sejam executadas ao mesmo tempo. Isso é útil para aproveitar melhor processadores com múltiplos núcleos.

Portanto, o POSIX se destina à padronização da interface entre aplicações e sistemas operacionais, promovendo compatibilidade, portabilidade e suporte ao desenvolvimento de programas concorrentes e paralelos.

b) OpenMP

OpenMP significa Open Multi-Processing. É uma API utilizada para programação paralela em sistemas de memória compartilhada, ou seja, sistemas nos quais várias threads acessam a mesma memória principal.

O OpenMP é usado em linguagens como C, C++ e Fortran. Ele permite que o programador paralelize partes de um programa de forma mais simples, utilizando diretivas, cláusulas, funções de biblioteca e variáveis de ambiente.

Uma das principais vantagens do OpenMP é sua facilidade de uso. Em vez de criar e controlar manualmente cada thread, como ocorre com pthreads, o programador pode inserir diretivas no código indicando quais trechos devem ser executados em paralelo. Por exemplo, em C ou C++, pode-se usar a diretiva `#pragma omp parallel for` para dividir as iterações de um laço entre várias threads.

O OpenMP é destinado a melhorar o desempenho de programas que executam tarefas repetitivas ou independentes, como cálculos científicos, processamento de matrizes, simulações, análise de dados e algoritmos numéricos. Ele é bastante usado em computadores pessoais, servidores e supercomputadores.

Assim, o OpenMP se destina à criação de aplicações paralelas de forma portátil, escalável e relativamente simples em arquiteturas de memória compartilhada.

Fonte: O site oficial do OpenMP afirma que a API OpenMP oferece suporte à programação paralela de memória compartilhada multiplataforma em C/C++ e Fortran, definindo um modelo portátil e escalável.

c) MPI

MPI significa Message Passing Interface, ou Interface de Passagem de Mensagens. É um padrão usado para programação paralela principalmente em sistemas de memória distribuída, como clusters e supercomputadores.

Diferentemente do OpenMP, em que as threads compartilham a mesma memória, no MPI cada processo geralmente possui seu próprio espaço de memória. A comunicação entre os processos ocorre por meio do envio e recebimento de mensagens.

O MPI permite que vários processos trabalhem juntos para resolver um problema maior. Cada processo pode executar uma parte da tarefa e trocar informações com os demais quando necessário. Essa comunicação pode ocorrer, por exemplo, por envio direto de mensagens entre dois processos ou por operações coletivas envolvendo vários processos.

O MPI é muito utilizado em computação de alto desempenho, simulações científicas, previsão do tempo, modelagem física, processamento de grandes volumes de dados e aplicações que precisam ser executadas em várias máquinas conectadas em rede.

Uma característica importante do MPI é que ele é um padrão, e não apenas uma biblioteca específica. Existem diferentes implementações de MPI, como MPICH e Open MPI, mas todas seguem a mesma especificação geral. Isso ajuda na portabilidade dos programas.

Portanto, o MPI se destina ao desenvolvimento de programas paralelos baseados em processos que se comunicam por mensagens, especialmente em ambientes de memória distribuída.

2) Explique detalhadamente SPMD e MPMD

SPMD — Single Program, Multiple Data

SPMD significa Single Program, Multiple Data, ou seja, Programa Único, Múltiplos Dados. Nesse modelo, todos os processadores ou processos executam o mesmo programa, mas cada um trabalha sobre uma parte diferente dos dados.

Mesmo que todos executem o mesmo código, eles podem seguir caminhos diferentes dentro do programa dependendo do identificador do processo, da thread ou da parte dos dados que receberam. Por exemplo, em um programa MPI, vários processos podem executar o mesmo arquivo executável, mas cada processo recebe um número de identificação chamado rank. Com base nesse rank, cada processo sabe qual parte do trabalho deve executar.

Um exemplo simples seria o processamento de um vetor grande. O mesmo programa é executado por vários processos, mas cada processo calcula apenas uma parte do vetor. Ao final, os resultados podem ser reunidos.

O modelo SPMD é muito comum em programação paralela porque simplifica o desenvolvimento: em vez de escrever vários programas diferentes, escreve-se um único programa capaz de se comportar de maneira diferente conforme o processo ou os dados atribuídos.

Esse modelo é bastante usado em MPI, OpenMP, processamento científico, simulações e computação de alto desempenho.

MPMD — Multiple Program, Multiple Data

MPMD significa Multiple Program, Multiple Data, ou seja, Múltiplos Programas, Múltiplos Dados. Nesse modelo, diferentes processadores ou processos podem executar programas diferentes, cada um trabalhando sobre dados diferentes.

Ao contrário do SPMD, em que todos executam o mesmo programa, no MPMD cada processo pode ter uma função específica. Por exemplo, em uma aplicação paralela, um programa pode ser responsável por ler os dados, outro por processá-los e outro por gravar ou exibir os resultados. Esses programas diferentes podem se comunicar entre si durante a execução.

Esse modelo é útil quando o problema pode ser dividido em etapas ou funções distintas. Em vez de todos os processos fazerem o mesmo tipo de tarefa, cada grupo de processos pode ser especializado em uma parte da aplicação.

Um exemplo seria uma simulação científica em que um programa calcula o comportamento físico de um sistema, outro programa realiza a visualização dos resultados e outro faz o armazenamento dos dados. Todos podem executar ao mesmo tempo e trocar informações.

O MPMD é mais flexível que o SPMD, pois permite combinar programas diferentes em uma mesma aplicação paralela. Porém, também pode ser mais complexo, já que exige maior cuidado na comunicação e coordenação entre os processos.

Portanto, o MPMD se destina a aplicações paralelas compostas por diferentes programas ou módulos, cada um executando tarefas específicas sobre conjuntos diferentes de dados.

Referências

IEEE. IEEE 1003.1-2024 — Portable Operating System Interface POSIX. Disponível em: IEEE Standards.

OpenMP. OpenMP API Specifications. Disponível em: site oficial OpenMP.

MPI Forum. Message Passing Interface Forum. Disponível em: site oficial MPI Forum.

Argonne National Laboratory. The Message Passing Interface MPI Standard. Disponível em: projeto MPI.

WIKIPEDIA. Single Program, Multiple Data. Disponível em: Wikipedia.

GeeksforGeeks. Single Program Multiple Data SPMD Model. Disponível em: GeeksforGeeks.